DEP-2 · 60 minutes

Plan it. Gate it. Ship it.

Infrastructure and pipeline as code with GitHub Copilot: Terraform on Azure, deployed by GitHub Actions

Wealth Accounts API (sample banking app) | Terraform · Azure App Service | AI-SDLC DevSecOps

DEP-2 A DIFFERENT RISK PROFILE

A bad suggestion can delete live infrastructure

TASK	WHERE COPILOT HELPS	WHERE A HUMAN STAYS IN THE LOOP
Draft new resources	A first draft from a plain-language brief	Naming, tags and sizing match team standards
Review a plan	Summarize the diff, flag destroys	Decide whether a replacement is actually safe
Pipeline as code	Scaffold stages and guardrails	Check it is compliant, not just working
`terraform apply`	Never	Always a human, behind an approval gate

> Copilot drafts infrastructure and explains plans. A human approves and applies.

Setup SAME REPO, NEW GUARDRAILS

The starter gives Copilot conventions to copy and rules to obey

infra/main.tf

```
locals {
  name_prefix =
"${var.app_name}-${var.environment}"
  tags = {
    app         = var.app_name
    environment = var.environment
    owner       = var.owner
    managed_by  = "terraform"
  }
}

resource "azurerm_resource_group" "main" {
  name     = "rg-${local.name_prefix}"
  location = var.location
```

Instructions

`terraform.instructions.md` and `workflows.instructions.md`: naming, tags, OIDC, approval gate

devops agent

Explains plans and CI failures. **Never applies.**

Prompt files

`/scaffold-infra` `/explain-plan` `/scaffold-pipeline` `/break-pipeline`

Step 1 AGENT MODE

A brief becomes Terraform that matches the file it lands in

Copilot Chat — Agent

```
/scaffold-infra spec=specs/INFRA.md
```

THE BRIEF ASKS FOR

- Linux B1 service plan
- Node 20 web app: HTTPS only, TLS 1.2, FTPS off
- a `transfer_limit` variable and `api_hostname`, `api_app_name` and `resource_group_name` outputs

1. Reads the **brief**, **main.tf** and the instructions
2. Lists the **conventions** it will follow, with citations
3. Shows a **short plan** and stops
4. You review it and reply **Approved**
5. Edits **main.tf** only
6. Runs **fmt** and **validate**, never apply

Step 1 CHECK THE RESULT

Valid is not enough: does it follow our conventions?

infra/main.tf · expected

```
resource "azurerm_linux_web_app" "api" {
  name                = "app-${local.name_prefix}-
api"
  resource_group_name =
azurerm_resource_group.main.name
  location            =
azurerm_resource_group.main.location
  service_plan_id     = azurerm_service_plan.main.id
  https_only          = true
  tags                = local.tags

  site_config {
    minimum_tls_version = "1.2"
    ftps_state          = "Disabled"
    application_stack { node_version = "20-lts" }
  }
```

Good signs

- Name uses `local.name_prefix`
- Tags only through `local.tags`
- Location comes from the resource group

Red flags

- A second `tags = { ... }` block
- A hard-coded region or name
- `node_version = "18-lts"`

Task: Infra: format and validate

Step 2 ASK MODE

Read the plan like the approver: look for -/+ first

Copilot Chat — Ask

```
/explain-plan plan=infra/samples/plan-forced-replacement.txt
```

A hand-written sample in real `terraform plan` format, so the demo needs no Azure subscription.

terraform plan

```
-/+ resource "azurerm_resource_group" "main" {
      ~ location = "canadacentral" -> "eastus" # forces replacement
    }
-/+ resource "azurerm_service_plan" "main" { ... }
-/+ resource "azurerm_linux_web_app" "api" { ... }

Plan: 3 to add, 0 to change, 3 to destroy.
```

Step 2 WHAT A GOOD REVIEW SAYS

One input change replaces everything in the resource group

3 / 3

Added / destroyed

Nothing is updated in place. Every resource is torn down and rebuilt.

`location`

The cause

The region default moved from `canadacentral` to `eastus`. The plan and web app follow the resource group.

Block

The recommendation

A region move needs a migration plan, not an in-place edit.

Copilot explains the plan. **It does not apply it.**

Step 3 AGENT MODE

The pipeline has five jobs and one approval gate

Copilot Chat — Agent

```
/scaffold-pipeline workflow=.github/workflows/infra.yml
```

EVERY RUN

validate

`fmt`, `init`, `validate`

→

PULL REQUESTS

plan

OIDC login, `plan`, posted as a PR comment

→

PUSH TO MAIN ONLY

apply

`environment: production` approval, re-plan, apply, then deploy `app/`. Then smoke, with rollback on demand.

Step 3 AUDIT THE DRAFT

A working pipeline is not a compliant pipeline

FIRST-DRAFT MISS	WHAT THE GUARDRAIL REQUIRES
Stored `ARM_CLIENT_SECRET`	OIDC with `azure/login@v2` and `id-token: write`
`-auto-approve`	Apply the reviewed `tfplan` file
No approval on apply	`environment: production` with reviewers
Write-all permissions	`contents: read`, extras per job
No `concurrency`	Two applies can't run at once

Copilot Chat Ask

```
Audit this workflow against
#file:.github/instructions/workflows.i
nstructions.md. What violates them?
```

Don't fix the draft for Copilot. **Make it find the problems.**

Step 3 AFTER APPLY

Every deploy ships with a smoke check and a way back

smoke

Calls `/api/health` on the deployed hostname and retries for two minutes. A failure fails the run.

rollback

Manual `workflow_dispatch` with a `rollback_ref`. Re-applies a known-good commit.

Same gate

Rollback runs behind the same `production` approval. It is the reviewed path, pointed at an older commit.

Step 4 CI FAILURE

The validate step fails in CI. Reproduce it, then ask why.

`terraform -chdir=infra validate`

```
Error: Reference to undeclared input variable

  on main.tf line 50, in locals:
  50:     environment = var.env

An input variable with the name "env" has not
been declared. This variable can be declared
with a variable "env" {} block.
```

Demo: break the pipeline

Run it in a regular terminal

devops agent

`#terminalLastCommand` This failed in CI. What is the root cause and the smallest fix?

Good fix

`var.env` becomes `var.environment`.

Weak fix

Declaring a new `env` variable creates two sources of truth.

Review DRAFT, NOT DONE

Generated code is a draft. You are accountable.

ALWAYS CHECK	IN THIS DEMO
Secrets	OIDC, no stored client secret
Authorization	Least-privilege `permissions`, approval gate
Personal data in logs	Plan and smoke output contain none
Error handling	Plan comment posted even on failure
Dependencies	Only approved actions, pinned versions

⚠ What Copilot cannot verify

- Your tag policy
- Approved regions
- Approved actions list
- Private networking rules

Ask it to **name what it could not verify**, then check those yourself.

TAKEAWAYS

Four habits for infrastructure with Copilot

Ground it

Prompt from the conventions already in the code: names, tags, location.

Read the plan

Look for -/+ lines before anything else.

Gate it

OIDC, approvals and no auto-approve live in the instructions, and Copilot audits itself.

Debug CI

Reproduce, evidence, root cause, smallest fix.

Copilot drafts. **A human applies.**



Thank you

Plan it. Gate it. Ship it.

docs.github.com/copilot

Questions?